

Jio Subscriber Retention & AI Chatbot Project

End-to-End Architecture Plan - Completion Report | Project-5 | Submitted 22 Sep 2026

Status: As-Built

Unlike a purely speculative plan, this document describes a system that has been fully built, run, and validated end-to-end across all four project phases: data analysis (Day 1), predictive modelling (Day 2), and a GenAI Text-to-SQL + RAG chatbot (Days 3-4). Every component named below corresponds to an actual file, database table, or verified test result in the project, not a proposed design.

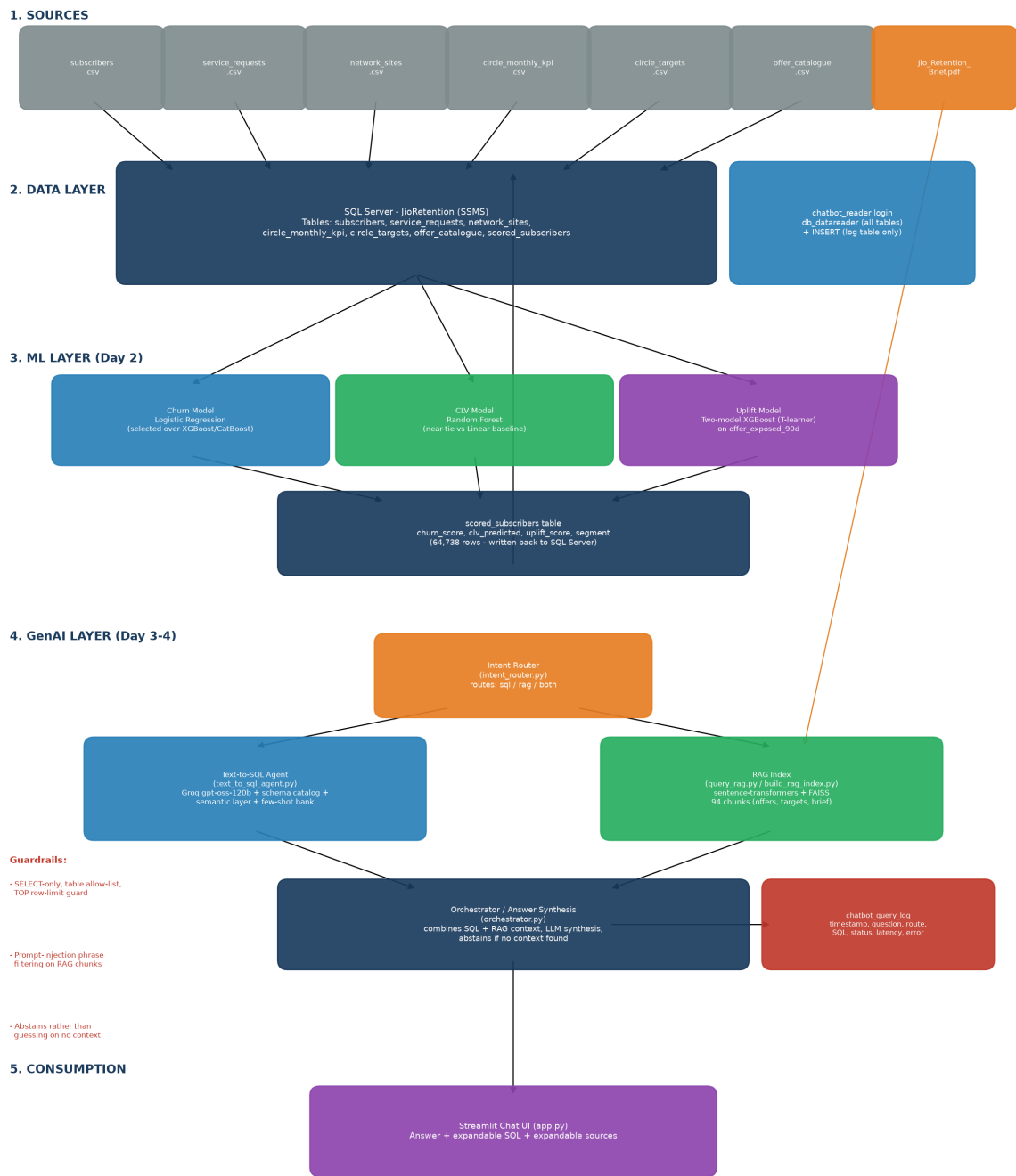


Figure 1 - Full system architecture, five layers: Sources, Data, ML, GenAI, Consumption

Layer-by-Layer Narrative

1. Sources

Six structured sheets (subscribers, service_requests, network_sites, circle_monthly_kpi, circle_targets, offer_catalogue) were exported from the original Jio_Retention_Dataset.xlsx workbook into individual CSVs for SQL Server import. The engagement brief PDF (Jio_Retention_Brief.pdf) follows a separate path - it is not structured data, so it bypasses the SQL data layer entirely and feeds directly into the RAG index.

2. Data Layer

A single SQL Server database (JioRetention, managed via SSMS) holds all seven structured tables, including scored_subscribers - the output of the ML layer, imported back in as an eighth data source. Access is via a dedicated chatbot_reader login scoped to the minimum privilege needed: db_datareader across all tables, plus a narrow INSERT grant on chatbot_query_log only, rather than blanket write access. One real data-quality issue was caught and fixed here during build: the Import Flat File wizard auto-typed service_requests.sr_id as MONEY, silently corrupting the text identifier (e.g. 'SR300001418' became '300001418.00') - resolved by re-importing with the column explicitly set to nvarchar(50).

3. ML Layer (Day 2)

Three independent models write their outputs into one combined scored_subscribers table (64,738 rows): a 30-day churn model (Logistic Regression, selected over XGBoost and CatBoost after 5-fold cross-validated comparison - CV PR-AUC 0.207 vs 0.178 vs 0.201 respectively), a CLV regression (Random Forest, a near-statistical-tie with a Linear Regression baseline, $R^2 \sim 0.58$), and a two-model (T-learner) uplift model separating persuadable subscribers (7.9% of the base) from sure-things/lost-causes (84.6%) and sleeping dogs (7.5%) using offer_exposed_90d as the treatment signal. This table is the single bridge between the ML and GenAI layers - the chatbot never calls a model directly, it only queries this pre-scored table.

4. GenAI Layer (Days 3-4)

An intent router (keyword- and pattern-based, including a regex check for subscriber-ID mentions) classifies each question as sql, rag, or both before any expensive work happens. The Text-to-SQL agent (Groq's openai/gpt-oss-120b model) is grounded with three things injected into every prompt: a live schema catalog introspected via SQLAlchemy, a certified semantic-layer glossary (so "churn," "ARPU," and "uplift segment" can't be redefined by the model), and a hand-written few-shot query bank. Generated SQL passes through a validator before execution: SELECT-only, a table allow-list, and an automatic TOP row-limit guard. In parallel, a RAG index (sentence-transformers embeddings over 94 chunks from the offer catalogue, circle targets, and the brief PDF, stored in FAISS) handles document-grounded questions. Both paths feed a final orchestrator that synthesizes one natural-language answer and explicitly abstains ("I don't have enough information") rather than fabricating an answer when no context was retrieved - demonstrated live when a test question about a Gujarat subscriber correctly reported no matching active offer existed, instead of inventing one. Every question, its route, generated SQL, execution status, row count, latency, and any error is written to chatbot_query_log with a timestamp, satisfying the homework's audit-logging requirement.

5. Consumption

A Streamlit chat UI (app.py) is the single entry point: it takes a plain-English question, calls the orchestrator, and displays the synthesized answer alongside two collapsible panels - the exact SQL that was generated and executed, and the raw retrieved document sources - so any answer can be independently checked rather than taken on faith.

Technology Stack (as implemented)

Layer	Component	Choice
Data	Database engine	SQL Server (via SSMS)
Data	Python-to-DB driver	SQLAlchemy + pyodbc
ML	Churn / CLV / Uplift	scikit-learn, XGBoost, CatBoost
ML	Feature/model storage	joblib (.pkl), CSV
GenAI	LLM (SQL generation + synthesis)	Groq API - openai/gpt-oss-120b
GenAI	Embeddings	sentence-transformers (all-MiniLM-L6-v2)
GenAI	Vector index	FAISS (local, in-process)
App	UI	Streamlit
App	Secrets management	python-dotenv (.env, gitignored)

Guardrails Implemented

- Read-only database access for the chatbot (db_datareader), with write access narrowed to a single INSERT grant on the log table only - never blanket db_datawriter.
- SQL validation before execution: hard block on DROP/DELETE/UPDATE/INSERT/ALTER/TRUNCATE/EXEC/MERGE/GRANT, a table allow-list, and an automatic row-limit guard.
- Prompt-injection phrase filtering on retrieved RAG content before it reaches the synthesis prompt.
- Explicit abstention ("I don't have enough information") in preference to a plausible wrong answer, when neither SQL nor RAG returns usable context.
- Full audit trail: every question, route, generated SQL, execution outcome, latency, and error is logged with a timestamp.

Validation

An 8-question golden test set covering all three routing paths (sql, rag, both) was run against the live system end-to-end: **8/8 passed** - correct routing decision and a substantive, non-abstaining answer for each. Individual component checks along the way (schema catalog introspection, RAG retrieval quality after the circle-code-to-name fix, subscriber-specific "both"-route reasoning) were verified against real, known values from the Day 1/Day 2 outputs rather than assumed correct.

Sources: scripts/day1_data_analysis.py, day2_*.py, build_schema_catalog.py, build_rag_index.py, text_to_sql_agent.py, intent_router.py, orchestrator.py, app.py, run_golden_tests.py; data/processed/scored_subscribers.csv, schema_catalog.json, golden_qa.json; SQL Server database JioRetention.